PRD: Tasks Screen

1. Screen Overview

The Tasks screen is the central task management hub for Studio OS. It lives at `/tasks` in the app sidebar navigation and gives studio leads, project managers, and designers a single place to view, manage, and track all work tasks across every project. The screen solves the problem of scattered task awareness — users can immediately see what is pending, in progress, waiting for review, or overdue, both for themselves and for the team. It is a high-frequency, daily-use screen and must be fast to scan and act on.

---

2. User Goals

- Quickly see all tasks that need attention today (overdue + due soon)
- Filter the task list to focus on their own tasks or a specific project
- Create a new task and assign it to the right person and project
- Change a task's status without leaving the list (inline status change)
- Know at a glance which tasks are overdue, high priority, or unassigned
- Mark a task as completed
- Open task details to view or edit the full record

---

3. Business Goals

- Make the studio's workload visible and organized across all projects
- Reduce missed deadlines by surfacing overdue and due-soon tasks prominently
- Support both personal productivity and team-level oversight in one screen
- Establish Tasks as a daily-use screen that keeps the studio operating efficiently

---

4. Primary Users

User	Context	Frequency
Studio lead / manager	Monitors all team tasks and workload	Daily
Project manager	Manages tasks scoped to specific projects	Daily
Designer / team member	Views and updates their own assigned tasks	Daily

The screen must support both a personal view (My Tasks) and a team-wide view (All Tasks).

---

5. Main Use Cases

UC-01 — View all tasks

Trigger: User navigates to `/tasks`.

Action: The screen loads with the All Tasks tab active, showing all tasks across projects.

Outcome: User sees a list of tasks with status, priority, project, assignee, and due date.

UC-02 — Switch to personal view

Trigger: User clicks the "My Tasks" tab.

Action: The list filters to show only tasks assigned to the current user.

Outcome: User sees only their own workload.

UC-03 — Filter tasks

Trigger: User opens the filter panel or selects a tab (To Do, In Progress, In Review, Completed).

Action: Active filters narrow the visible task list. Multiple filters compose together.

Outcome: Only tasks matching all active filters are shown.

UC-04 — Search tasks

Trigger: User types in the search input.

Action: List filters in real time by task title.

Outcome: Only tasks whose title matches the query are shown.

UC-05 — Create a new task

Trigger: User clicks "New Task" button.

Action: A dialog opens with fields for title, status, priority, project, assignee, due date, and optional description.

Outcome: New task appears at the top of the list; stat counters update.

UC-06 — Change task status inline

Trigger: User clicks the status badge on a task row.

Action: A dropdown opens with the four status options.

Outcome: Status updates immediately in the row; no page reload.

UC-07 — Mark task as completed

Trigger: User checks the completion checkbox on a task row.

Action: Task status changes to Completed; row updates visually (muted text, strikethrough title).

Outcome: Task moves to Completed count; still visible in All Tasks unless filtered out.

UC-08 — Open task details

Trigger: User clicks on a task row (outside the checkbox and status badge).

Action: A detail Sheet slides in from the right with the full task record.

Outcome: User can read or edit all task fields in the Sheet.

UC-09 — Sort tasks

Trigger: User selects a sort option (Due date, Priority, Recently updated, Project).

Action: List reorders by the selected field.

Outcome: Tasks are displayed in the new sort order.

---

6. Functional Requirements

FR-01 — The screen renders a list of all tasks, defaulting to the All Tasks tab, sorted by due date ascending.

FR-02 — Tabs filter the task list: All Tasks · My Tasks · To Do · In Progress · In Review · Completed. Active tab is visually highlighted.

FR-03 — A search input filters tasks by title in real time (client-side, case-insensitive).

FR-04 — A filter panel (Popover, matching the Projects screen pattern) supports composable filters: Status · Priority · Project · Assignee · Due date range. Active filter count is shown as a badge on the filter button.

FR-05 — A "Clear all filters" action resets all active filters and search.

FR-06 — A sort selector allows reordering by: Due date · Priority · Recently updated · Project.

FR-07 — Each task row displays: completion checkbox · title · status badge · priority badge · project name · assignee avatar · due date · row action menu (MoreHorizontal).

FR-08 — Clicking the status badge on a row opens a DropdownMenu with the four status options. Selecting an option updates the row's status immediately (local state).

FR-09 — Clicking the completion checkbox sets the task status to Completed and applies visual treatment (muted text, title strikethrough).

FR-10 — Clicking a task row (excluding checkbox and status badge) opens a Sheet from the right with the full task detail view.

FR-11 — The "New Task" button opens a Dialog with fields: title (required), status (required), priority (required), project (required), due date (required), assignee (optional), description (optional), tags (optional). The Create button is disabled until required fields are filled.

FR-12 — Overdue tasks (due date < today and status ≠ Completed) are visually flagged: due date shown in `text-red-500`, an overdue label or icon.

FR-13 — A summary stats row at the top of the screen shows: Total · To Do · In Progress · In Review · Completed counts, calculated from the current unfiltered task list.

FR-14 — When filters or search return no results, a no-results empty state is shown with copy and a "Clear filters" action.

FR-15 — When there are no tasks at all, a zero-data empty state is shown with copy and a "New Task" CTA.

---

7. Content Requirements

Task fields:

Field	Type	Required	Notes
id	string	yes	Generated on create
title	string	yes	Max display ~60 chars; truncate with tooltip
description	string	no	Short context; shown in detail Sheet
status	enum	yes	To Do · In Progress · In Review · Completed
priority	enum	yes	Low · Medium · High · Urgent
project	string	yes	Project name; links visually to Projects screen
assignee	{ name, avatar }	no	Single assignee; show avatar + name
dueDate	ISO date string	yes	Display as "Apr 12"; derive overdue flag
createdAt	ISO date string	yes	For "Recently updated" sort
tags	string[]	no	Optional labels; display as small pills

Status labels and colors:

Status	Color treatment
To Do	`bg-muted text-muted-foreground`
In Progress	`bg-amber-100 text-amber-700 dark:bg-amber-900/30 dark:text-amber-400`
In Review	`bg-blue-100 text-blue-700 dark:bg-blue-900/30 dark:text-blue-400` (use Tailwind `blue-*`)
Completed	`bg-emerald-100 text-emerald-700 dark:bg-emerald-900/30 dark:text-emerald-400`

Priority labels and colors:

Priority	Color treatment
Low	`bg-muted text-muted-foreground`
Medium	`bg-amber-100 text-amber-700 dark:bg-amber-900/30 dark:text-amber-400`
High	`bg-orange-100 text-orange-700 dark:bg-orange-900/30 dark:text-orange-400`
Urgent	`bg-red-100 text-red-700 dark:bg-red-900/30 dark:text-red-400`

Empty state copy:

- Zero tasks: "No tasks yet. Create your first task to get started." + "New Task" button
- No results: "No tasks match your current filters." + "Clear filters" link

Overdue label: "Overdue" — shown next to or replacing the due date, in `text-red-500`.

---

8. UX Requirements

- **Tab + filter composition:** Tabs act as a quick status filter. Additional filters in the Popover narrow within the active tab. Both work together — switching a tab does not reset Popover filters.
- **Inline status editing:** Clicking the status badge opens a DropdownMenu directly on the row (no Sheet needed for a status-only change). This mirrors the inline status editing pattern in `all-projects-table.tsx`.
- **Completion checkbox:** Visually prominent; checking it immediately applies a strikethrough + muted style to the row title and updates status to Completed.
- **Sheet for task details:** Opening a task row slides a Sheet in from the right without navigating away. The Sheet shows all fields and allows editing. Closing the Sheet returns focus to the list.
- **Search:** Input is always visible in the toolbar, not behind a toggle. Searching does not reset the active tab.
- **Filter badge:** The filter button shows a count badge when filters are active, matching the Projects screen Popover pattern exactly.
- **Sort:** A Select component in the toolbar. Default: Due date ascending. Sort persists while filters/tabs change.

- **Overdue visibility:** Overdue tasks are not hidden or moved — they stay in the list, flagged visually. They appear prominently in the To Do and In Progress tabs.
- **New Task dialog:** Follows the same Dialog pattern as `new-project-dialog.tsx`. Required field validation prevents submission of incomplete tasks.

---

9. UI Requirements

- **Page header:** Same pattern as Projects screen — title ("Tasks") + subtitle + action buttons (Filter, Sort, New Task) right-aligned.
- **Stats row:** Four stat Cards below the header, matching `overview-stats.tsx` layout from the Projects screen. Cards show: Total, To Do, In Progress, Completed.
- **Tabs:** Use the `Tabs / TabsList / TabsTrigger / TabsContent` primitive from `components/ui/tabs.tsx`. Tabs sit between the stats row and the task list.
- **Task list:** A table using `components/ui/` Table primitives (or a styled row list matching the Projects table aesthetic). Columns: [ ] · Title · Status · Priority · Project · Assignee · Due Date · Actions.
- **Status badge:** Inline pill, same shape and pattern as `HealthBadge` in `components/projects/health-badge.tsx` — `rounded-full px-2.5 py-0.5 text-xs font-medium`.
- **Priority badge:** Same shape as status badge, using the priority color table above.
- **Assignee:** `Avatar` + name, using `components/ui/avatar.tsx` at `size="sm"`.
- **Due date:** Plain text in `text-sm text-muted-foreground`; overdue in `text-sm text-red-500 font-medium`.
- **Row action menu:** `MoreHorizontal` icon button opening a `DropdownMenu` with: Edit, Change Status, Assign, Set Due Date, Delete (destructive).
- **Sheet (task detail):** Uses `Sheet / SheetContent / SheetHeader / SheetTitle` from `components/ui/sheet.tsx`, side="right", width ~480px.
- **New Task dialog:** Uses `Dialog` from `components/ui/dialog.tsx`, same pattern as `new-project-dialog.tsx`.
- **Filter Popover:** Reuses the exact Popover + pill-toggle pattern from `app/projects/page.tsx` (client/member filter). Extend with Status, Priority, Due date fields.
- **Responsive:** Single column on mobile (hide Project and Assignee columns below `sm:`). Full table at `md:` and above.

---

10. Components Needed

Existing (reuse):

Component	Location	Used for
`Tabs, TabsList, TabsTrigger, TabsContent`	`components/ui/tabs.tsx`	Tab navigation
`Card, CardHeader, CardTitle, CardContent`	`components/ui/card.tsx`	Stats row cards
`Button`	`components/ui/button.tsx`	New Task, filter, sort buttons
`Input`	`components/ui/input.tsx`	Search field
`Select, SelectTrigger, SelectContent, SelectItem`	`components/ui/select.tsx`	Sort selector, status/priority in dialog
`Avatar, AvatarImage, AvatarFallback`	`components/ui/avatar.tsx`	Assignee display
`Sheet, SheetContent, SheetHeader, SheetTitle`	`components/ui/sheet.tsx`	Task detail panel
`Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter`	`components/ui/dialog.tsx`	New Task dialog
`DropdownMenu, DropdownMenuTrigger, DropdownMenuContent, DropdownMenuItem`	`components/ui/dropdown-menu.tsx`	Row actions, inline status change
`Separator`	`components/ui/separator.tsx`	Filter panel sections
`Tooltip, TooltipTrigger, TooltipContent`	`components/ui/tooltip.tsx`	Long title truncation, badge explanations
`Skeleton`	`components/ui/skeleton.tsx`	Loading state rows
`Popover, PopoverTrigger, PopoverContent`	`components/ui/popover.tsx`	Filter panel
`OverviewStats` pattern	`components/projects/overview-stats.tsx`	Stats row (adapt, don't import directly)
Filter Popover pattern	`app/projects/page.tsx`	Filter panel behavior
Inline status dropdown pattern	`components/projects/all-projects-table.tsx`	Inline status editing

New (build):

Component	Description	Primitives used
`TaskRow`	Single task row with checkbox, badges, avatar, due date, actions	`Avatar`, `DropdownMenu`, `Tooltip`, `Checkbox`
`TaskStatusBadge`	Inline status pill with color map	`cn()`, status color map
`TaskPriorityBadge`	Inline priority pill with color map	`cn()`, priority color map
`TaskStatsRow`	Four stat cards derived from task list	`Card`, `CardHeader`, `CardContent`
`NewTaskDialog`	Controlled dialog to create a new task	`Dialog`, `Input`, `Select`, `Button`
`TaskDetailSheet`	Full task detail / edit panel	`Sheet`, `Input`, `Select`, `Avatar`
`TaskEmptyState`	Zero-data and no-results empty states	`Button`
`TasksTable`	Table wrapper managing sort, row render	`TaskRow`, Table primitives

---

11. States

Task list — default:

Tasks are loaded, tabs and filters are at their defaults (All Tasks, no active filters), sorted by due date ascending.

Task list — loading:

Skeleton rows replace the task rows. Stats row shows skeleton cards. Tabs are visible but disabled.

Task list — empty (no tasks at all):

Center-aligned empty state: icon, "No tasks yet", description, "New Task" button.

Task list — no results (filter/search active):

"No tasks match your current filters." with a "Clear filters" text link. Filter badge remains visible.

Task row — overdue:

Due date shown in `text-red-500 font-medium`. Optional "Overdue" label. Row is otherwise unchanged.

Task row — completed:

Checkbox checked. Title has `line-through text-muted-foreground`. Status badge shows Completed (emerald).

Task row — missing assignee:

Assignee cell shows "—" or a blank avatar placeholder. No error state.

Task row — missing due date:

Due date cell shows "—". No overdue flag.

Task row — long title:

Title truncates with `truncate` CSS. Full title visible in `Tooltip` on hover.

New Task dialog — validation:

Create button disabled until title, status, priority, project, and due date are filled.

Sheet — editing:

Fields are editable inputs. Save / Cancel buttons in the Sheet footer.

---

12. Edge Cases

- **All tasks completed:** Stats show 0 active tasks. The To Do, In Progress, In Review tabs show empty states. Completed tab shows the full list.
- **Very large dataset (50+ tasks):** No pagination specified — use virtual scrolling or accept long scroll. Flag this as a scaling decision before implementation.
- **Multiple filters active with no results:** All active filter badges remain visible; no-results state is shown.
- **Task with no project assigned:** Show "—" in the Project column. Project filter does not include this task under any project option.
- **Due date = today:** Not overdue; display normally. Consider adding a "Due today" indicator (subtle amber text) if desired — this is optional.
- **Tab + filter conflict:** If the active tab is "In Progress" and the Status filter is set to "Completed," the result is always empty. This is valid behavior — show the no-results state.
- **"My Tasks" tab with no current user context:** Since the app is static and has no auth, "My Tasks" can be hardcoded to a specific mock user (e.g., Daniel) for now.
- **Deleting a task:** Removes it from the list. Stats update. No confirmation dialog required for now (can be added later).

---

13. Success Criteria

1. All tasks from `data.ts` are visible in the All Tasks tab by default, with correct status, priority, project, assignee, and due date displayed.
2. Tab switching correctly filters the task list (e.g., clicking "In Progress" shows only in-progress tasks).
3. Filters compose correctly — activating two or more filters narrows the list to tasks matching all active conditions.
4. Inline status change via the status badge dropdown updates the row and the stats row without a page reload.
5. The New Task dialog creates a task that immediately appears in the list and is reflected in the stats row counts.

6. Overdue tasks (past due date, not completed) are visually flagged in the list.
7. The no-results empty state is shown when filters/search return zero tasks; the zero-data empty state is shown when no tasks exist.

---

14. Implementation Notes

Routing: Page at `app/tasks/page.tsx`. Add `"use client"` directive (state-heavy screen).

Folder structure:

```
components/tasks/
├── tasks-table.tsx       — table wrapper, manages sort + row render
├── task-row.tsx          — single row component
├── task-status-badge.tsx — status pill (color map + cn())
├── task-priority-badge.tsx — priority pill
├── task-stats-row.tsx    — four stat cards derived from task list
├── new-task-dialog.tsx   — controlled dialog
├── task-detail-sheet.tsx — full detail/edit panel
├── task-empty-state.tsx  — zero-data and no-results variants
├── types.ts              — Task, TaskStatus, TaskPriority types
└── data.ts               — mock Task[] array (10–15 tasks)
```

TypeScript types (`types.ts`):

```
export type TaskStatus = "To Do" | "In Progress" | "In Review" | "Completed"
export type TaskPriority = "Low" | "Medium" | "High" | "Urgent"

export interface TaskAssignee {
  name: string
  avatar?: string
  initials: string
}

export interface Task {
  id: string
  title: string
  description?: string
  status: TaskStatus
  priority: TaskPriority
  project: string
  assignee?: TaskAssignee
  dueDate: string         // ISO date string
  createdAt: string       // ISO date string
  tags?: string[]
}
```

Mock data (`data.ts`): Create 12–15 tasks spread across all four statuses, multiple projects (reuse project names from `danielProjects` in `components/projects/data.ts`), multiple assignees (reuse team member names), a mix of priorities, and at least 2–3 overdue tasks. **State shape in `app/tasks/page.tsx` :**

```
const [tasks, setTasks] = useState<Task[]>(mockTasks)
const [activeTab, setActiveTab] = useState<string>("all")
const [search, setSearch] = useState("")
const [sortBy, setSortBy] = useState<SortKey>("dueDate")
const [filters, setFilters] = useState<FilterState>(EMPTY_FILTERS)
```

"My Tasks" tab: Hardcode to assignee name `"Daniel"` for the static mock. When auth is added later, replace with the current user's name. **No backend:** All interactions are local state only. No fetchers, no API calls, no server actions. **`dropdown-menu.tsx` primitive:** Verify this exists in `components/ui/` before implementation. If missing, add it via shadcn CLI or create following the same Radix pattern as `dialog.tsx` . **Checkbox:** `components/ui/` may not have a Checkbox primitive. Verify before implementing `TaskRow` . If missing, either add via shadcn or use a styled `<input type="checkbox">` with `cn()` .